

Writeup: tralalelotralalaOK

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_07_tralalelotralalaOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `niquelecasinoabc`

Résumé

WRITEUP

Reconnaissance rapide

```
file ; strings ; objdump -d -M intel
```

On repère `init_tables`, `transform_byte`, et les messages Good Job! / Bad Password!.

```
strings tralalelotralala | less
```

```
objdump -d -M intel tralalelotralala | less
```

```
readelf -S tralalelotralala
```

```
xxd -s 0x2000 -l 64 tralalelotralala
```

Repérage des "seeds"

Dans `.data` à `0x402000` : quatre blocs de 16 octets.

Hex-dump via `xxd` ; on les appelle `seed1..4`

`init_tables` = XOR constants

`key_xor` = `seed1 ^ 0x55`

`key_rot` = `seed2 ^ 0x33`

`key_add` = `seed3 ^ 0xAA`

`enc_expected` = `seed4 ^ 0x3C`

`transform_byte` (vue ASM)

`out = rol( (inp ^ key_xor[i]) , key_rot[i] & 7 ) + key_add[i]`

Inversion en Python(16 octets)

```
def ror(b,n): return ((b>>n)|(b<<(8-n))) & 0xFF
flag = bytes(
    ( (enc_expected[i]-key_add[i]) & 0xFF ) ^
    key_xor[i] if False else 0 #xor après rotation
    for i in range(16)
)
```

cela donne niquelecasinoabc

Outils Utilisés

```
file ; strings ; objdump -d -M intel
```

On repère init_tables, transform_byte, et les messages Good Job! / Bad Password!.

Analyse Statique

```
file ; strings ; objdump -d -M intel
```

On repère init_tables, transform_byte, et les messages Good Job! / Bad Password!.

Analyse Dynamique

L'exécution du programme a permis d'observer son comportement et d'identifier les points de validation.

Identification du Mécanisme de Validation

Validation

```
./tralalelotralala  
niquelecasinoabc  
Cela donne Good Job!
```

Découverte du Flag

```
file ; strings ; objdump -d -M intel
```

On repère init_tables, transform_byte, et les messages Good Job! / Bad Password!.

Conclusion

Ce crackme a été résolu en comprenant son mécanisme de validation et en élaborant une stratégie appropriée.